Machine Learning Operations (AIMLCZG523)

Assignment 01 — Heart Disease Risk Prediction

End-to-end ML development, CI/CD and production deployment

Name: VIPIN VK

BITS ID: 2025ae05835

Repository: https://github.com/2025ae05835/heart-mlops

Deployed API / access: Local Kubernetes (Minikube), not public: POST http://<EXTERNAL-IP>/predict after scripts/deploy_k8s.sh and minikube tunnel (Section 8, README section 5)

Video walkthrough: video/2025ae05835.mp4 in the repository

Date: October 2026

Contents

1. Overview and architecture
2. Setup and installation
3. Data acquisition and cleaning
4. Exploratory data analysis
5. Feature engineering and model development
6. Experiment tracking with MLflow
7. Packaging, testing and CI/CD
8. Containerisation and deployment
9. Monitoring and logging
10. Limitations and next steps
11. Links

1. Overview

The goal is a classifier that estimates the probability that a patient has heart disease from 13 routine clinical measurements, delivered as a reproducible, tested, containerised and monitored REST API. Every stage — data, training, testing, packaging, deployment and monitoring — is scripted and runs from a clean checkout using requirements.txt.

Requirement	Implementation
Data + EDA	heart.data (download/clean), heart.eda, notebooks/01_eda.ipynb
Models + tuning	LogReg, Random Forest, HistGradientBoosting; GridSearchCV, stratified 5-fold CV
Experiment tracking	MLflow: params, CV + hold-out metrics, plots, models, model registry
Packaging	Single sklearn Pipeline (preprocessing + model) → joblib + MLflow model
CI/CD	GitHub Actions: lint → test → train + quality gate → docker build/smoke → GHCR
Serving	FastAPI /predict (+ /predict/batch, /health, /metrics) in a non-root Docker image
Deployment	Kubernetes Deployment (2 replicas, probes, HPA), LoadBalancer + Ingress
Monitoring	Structured JSON request logs, Prometheus metrics, provisioned Grafana dashboard

Architecture

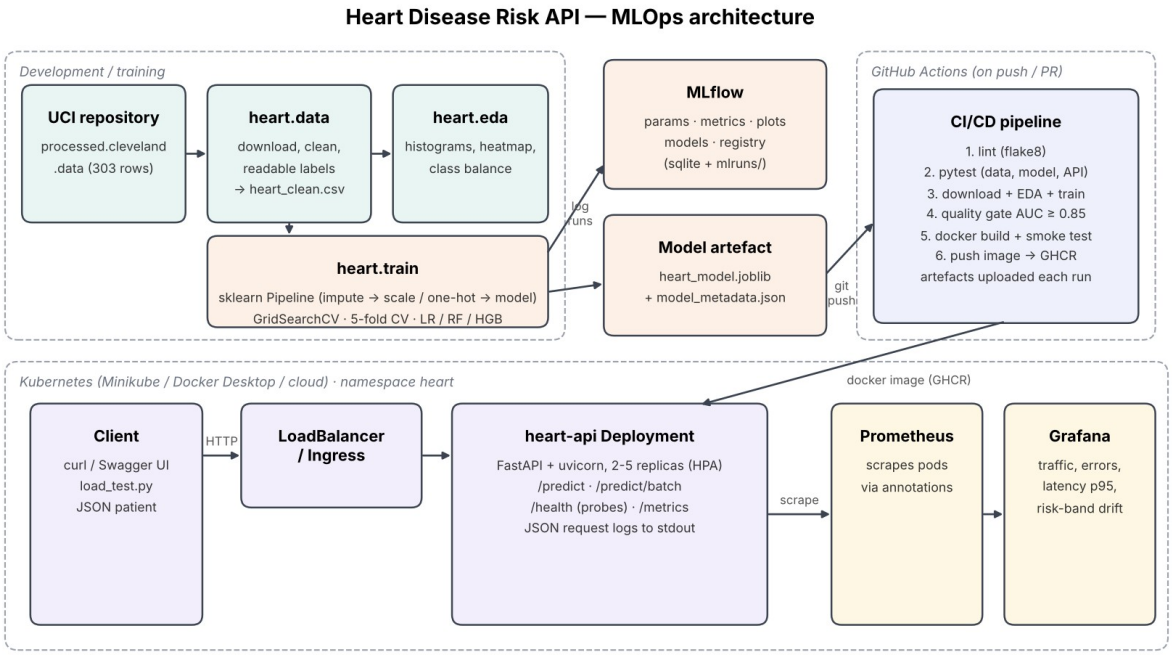


Figure 1. System architecture

2. Setup and installation

Prerequisites: Python 3.11+ (3.12 used in CI and Docker), Git, Docker, and Minikube or Docker Desktop Kubernetes. The project was developed and verified in this environment:

Component	Version / details
Lab VM	Rocky Linux 9.5, 4 vCPU, 15 GB RAM (BITS Prayogshala)

Component	Version / details
Python	3.12.15 in a conda environment kept on the persistent workspace disk
Docker	28.0.1
Kubernetes	Minikube 1.35 (docker driver) running Kubernetes 1.32, kubectl 1.32
CI	GitHub Actions, ubuntu-latest runners, Python 3.12
Source control	Git; pushes from both the lab VM and a Windows laptop to one GitHub repository

```
git clone https://github.com/2025ae05835/heart-mlops.git && cd heart-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export PYTHONPATH=src:.
python -m heart.data --force   # download + clean
python -m heart.eda            # figures
python -m heart.train          # tune, compare, log to MLflow, save model
pytest -v && flake8 src api test
uvicorn api.main:app --port 8000
```

A Makefile wraps these steps (make all). requirements.txt pins every version; the serving image uses the smaller requirements-serve.txt (no MLflow or plotting libraries), which keeps the image small and the attack surface low.

3. Data acquisition and cleaning

Source: UCI Machine Learning Repository, Heart Disease dataset, Cleveland subset (processed.cleveland.data, 303 patients, 13 features + diagnosis). The download script fetches the official file. Because the UCI server is occasionally unreachable, a fallback downloads a public mirror of the same 303 records; that mirror uses a different integer coding for cp, restecg, slope, thal and an inverted target, so the code translates it back to the UCI coding (verified row-by-row and covered by a unit test).

Cleaning decisions:

- **Target:** the original num (0–4) is binarised to presence (num > 0) vs absence of disease.
- **Missing values:** ca (4 rows) and thal (2 rows) are '?' in the source. They are kept and imputed inside the model pipeline (median / most frequent) so training and serving apply identical logic.
- **Plausibility:** zero or negative blood pressure, cholesterol or heart rate are treated as missing rather than real values.
- **Readable categories:** nominal codes are mapped to labels (e.g. cp 4 → asymptomatic). This makes the API self-documenting and prevents the model from treating codes as ordered numbers.
- **Duplicates:** the official file contains no exact duplicates, so all 303 patients are kept (164 no disease, 139 disease).

4. Exploratory data analysis

The classes are roughly balanced (54% / 46%), so accuracy is informative, but ROC-AUC is used for model selection because it is threshold-independent and reflects ranking of patients by risk.

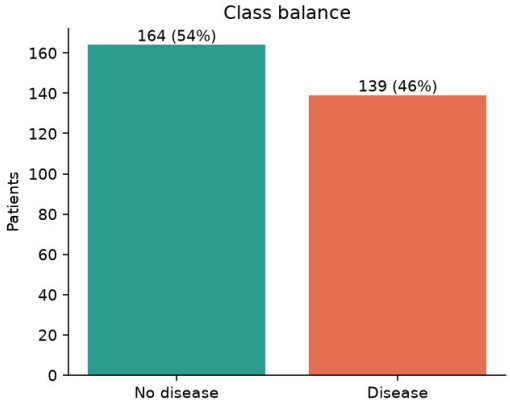


Figure 2. Class balance

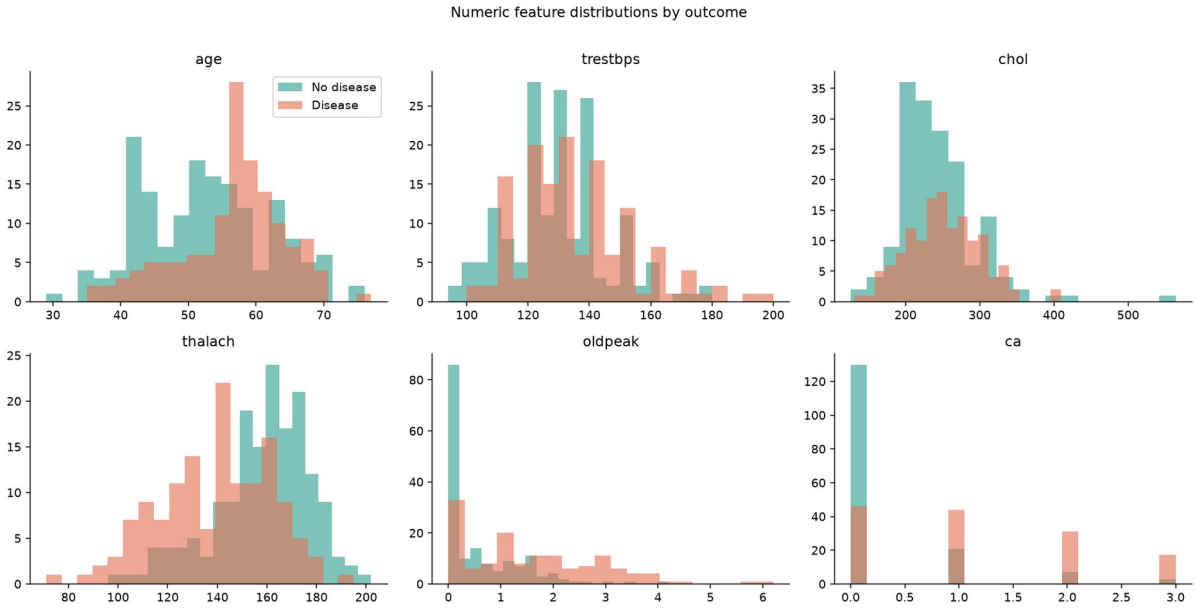


Figure 3. Numeric features by outcome

Patients with disease have markedly lower maximum heart rate (thalach), higher exercise-induced ST depression (oldpeak) and more coloured vessels (ca). Cholesterol and resting blood pressure separate the classes only weakly.

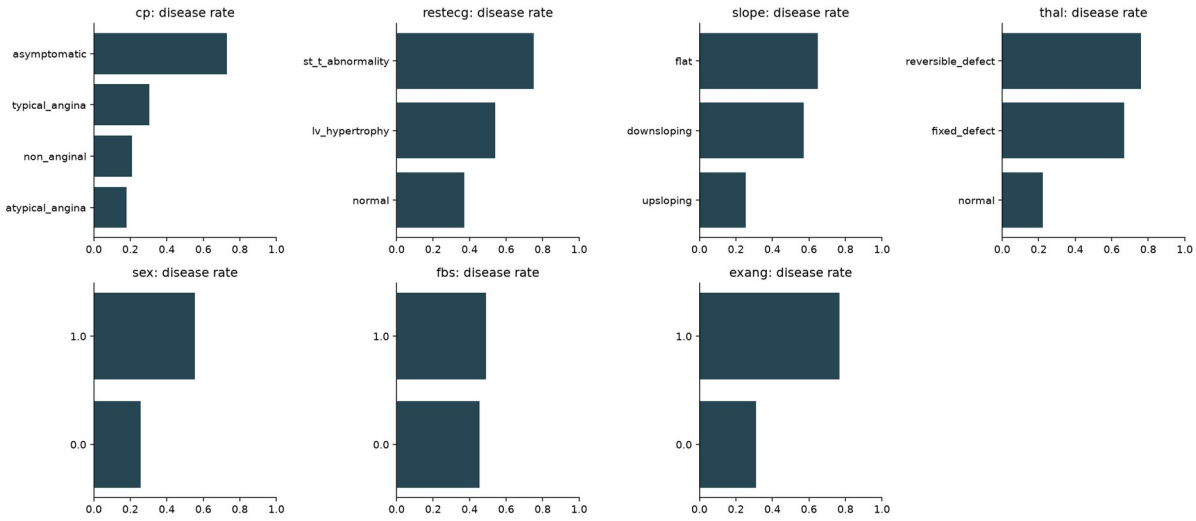


Figure 4. Disease rate per category

Asymptomatic chest pain (≈72% disease rate), reversible thalassemia defect, flat ST slope and exercise-induced angina are the strongest categorical signals.

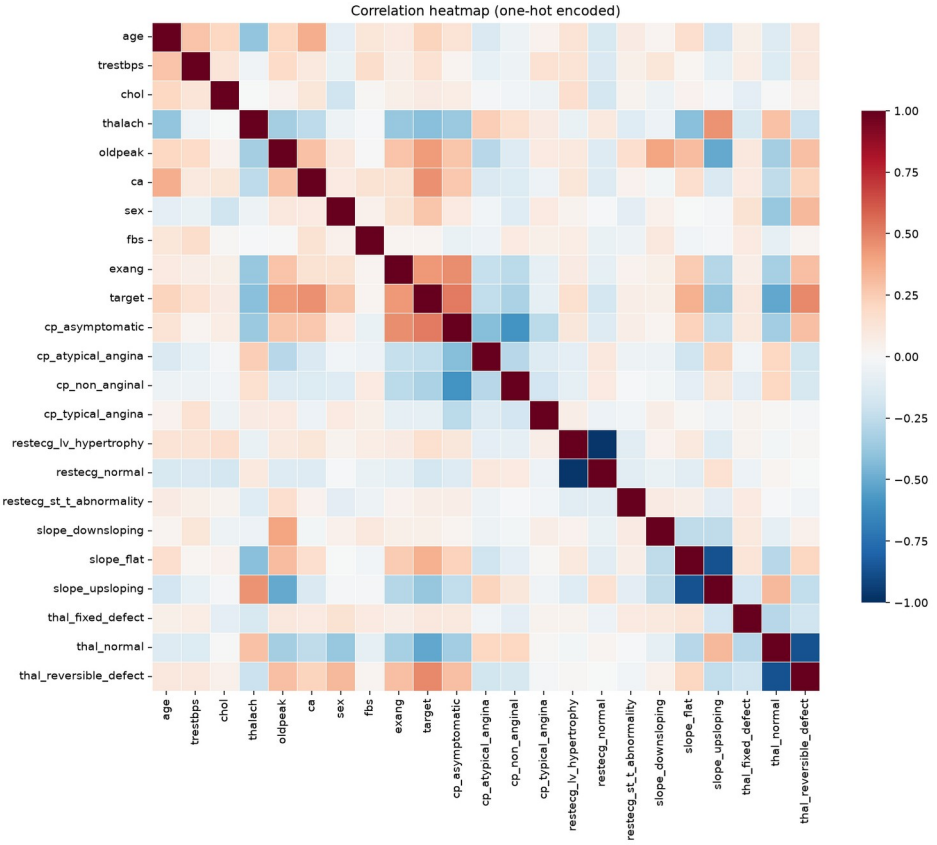


Figure 5. Correlation heatmap (one-hot encoded)

Feature	Correlation with target	Feature	Correlation with target
cp_asymptomatic	+0.52	thal_normal	−0.52
thal_reversible_defect	+0.48	thalach	−0.42
ca	+0.46	slope_upsloping	−0.39
exang	+0.43	cp_non_anginal	−0.32
oldpeak	+0.43	cp_atypical_angina	−0.25

No pair of input features is so strongly correlated that one should be dropped, so all 13 features are kept.

5. Feature engineering and model development

Preprocessing pipeline

A scikit-learn ColumnTransformer handles three feature groups and is fitted inside each CV fold, which prevents information from the validation fold leaking into scaling or imputation:

- Numeric (age, trestbps, chol, thalach, oldpeak, ca): median imputation → StandardScaler.
- Binary (sex, fbs, exang): most-frequent imputation, passed through.
- Nominal (cp, restecg, slope, thal): most-frequent imputation → one-hot encoding with handle_unknown='ignore', so an unseen category at serving time does not crash the model.

Models and tuning

The data is split 80/20 (stratified, seed 42). Each candidate is tuned with GridSearchCV over stratified 5-fold CV on the training part, optimising ROC-AUC; the tuned configuration is then re-scored with the same folds on accuracy, precision, recall, F1 and ROC-AUC.

Model	Grid size	Best parameters
Logistic Regression	24	C = 0.3, L2 (l1_ratio = 0), class_weight = balanced
Random Forest	48	200 trees, max_depth = 5, min_samples_leaf = 3, max_features = sqrt
HistGradientBoosting	16	learning_rate = 0.03, max_depth = 2, 100 iterations, L2 = 1.0

Results

Model	CV ROC-AUC	CV Acc.	CV F1	Test ROC-AUC	Test Acc.	Test Recall
Logistic Regression	**0.907 ± 0.019**	**0.855**	**0.834**	**0.961**	**0.869**	**0.929**
Random Forest	0.902 ± 0.021	0.814	0.785	0.955	0.885	0.929
HistGradientBoosting	0.894 ± 0.025	0.834	0.813	0.948	0.885	0.893

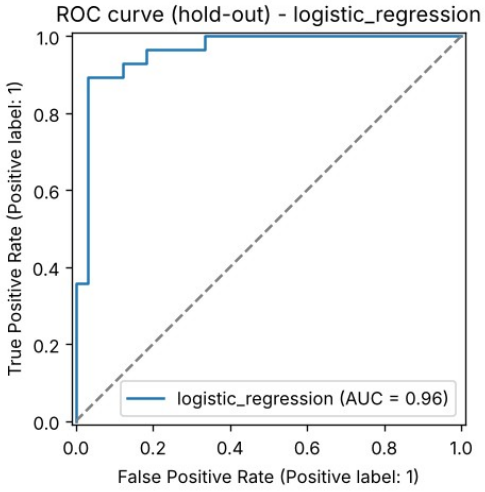


Figure 6. Hold-out ROC curve, Logistic Regression

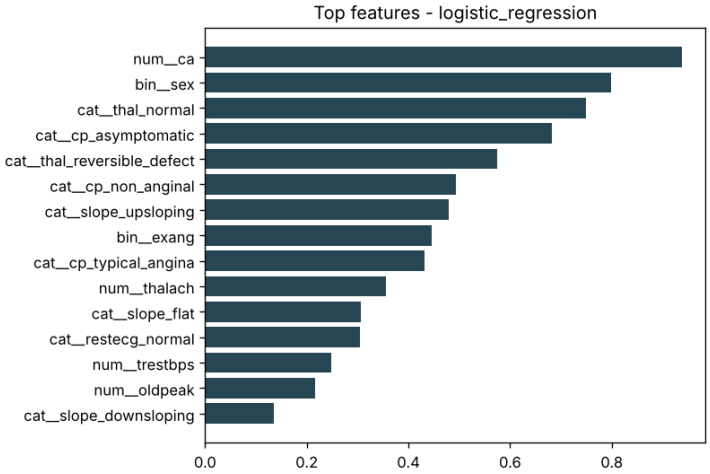


Figure 7. Largest absolute coefficients, Logistic Regression

Model selection

Logistic Regression has the best mean CV ROC-AUC (0.907) and the lowest variance across folds, and it also has the best hold-out ROC-AUC (0.961), so it was selected. Random Forest and Gradient Boosting reach slightly higher hold-out accuracy (0.885 vs 0.869), but with only 61 test patients that gap is a single patient, while ROC-AUC measures how well each model ranks every patient by risk. The three models are within about one standard deviation of each other, so the simplest model wins: its coefficients can be read directly (Figure 7): the largest belong to the number of coloured vessels, sex, the thalassemia result and asymptomatic chest pain, the same signals the EDA highlighted. class_weight = balanced favours recall, which reaches 0.929 on the hold-out. The selected configuration is finally refit on all 303 rows to produce the deployable model.

For a screening use case recall (catching true disease) matters more than precision; a natural next step is to lower the decision threshold below 0.5 and pick it from the precision-recall curve. The API already returns the raw probability and a risk band (low < 0.4 ≤ moderate < 0.7 ≤ high) so a clinician-facing client can apply its own threshold.

6. Experiment tracking with MLflow

MLflow uses a local SQLite backend (mlflow.db), which also enables the model registry. Every model family is one run in experiment heart-disease, plus a final run for the refit model:

- Parameters: best hyperparameters, CV folds, test size, training rows; tags for model family, grid size and quick-mode.
- Metrics: mean and standard deviation of CV accuracy, precision, recall, F1, ROC-AUC; hold-out metrics; training time.
- Artifacts: ROC curve, confusion matrix, feature-importance chart, full GridSearchCV results CSV, model comparison table, model metadata.
- Models: each tuned pipeline with signature and input example; the final model is registered as heart-disease-classifier (version per training run).

MLflow 3 serialises scikit-learn models with skops instead of pickle. skops refuses to load object types it cannot verify, so the training code explicitly trusts only the three tree/dtype types produced by our own runs rather than switching back to unrestricted pickle.

Figure 8 shows the experiment after two separate training sessions. Every metric of the second session is identical to the first (for example CV ROC-AUC 0.90749 for Logistic Regression in both), which confirms that the pipeline is deterministic. The final_logistic_regression runs carry the registered model (heart-disease-classifier, versions 1 and 2) rather than metrics.

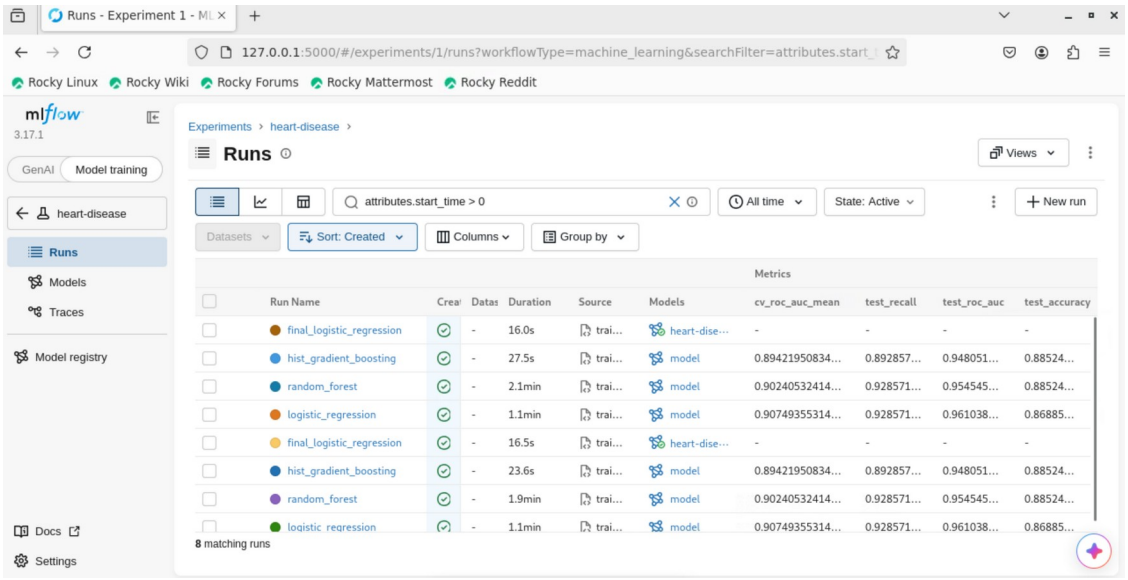


Figure 8. MLflow runs with CV and hold-out metrics (two identical training sessions)

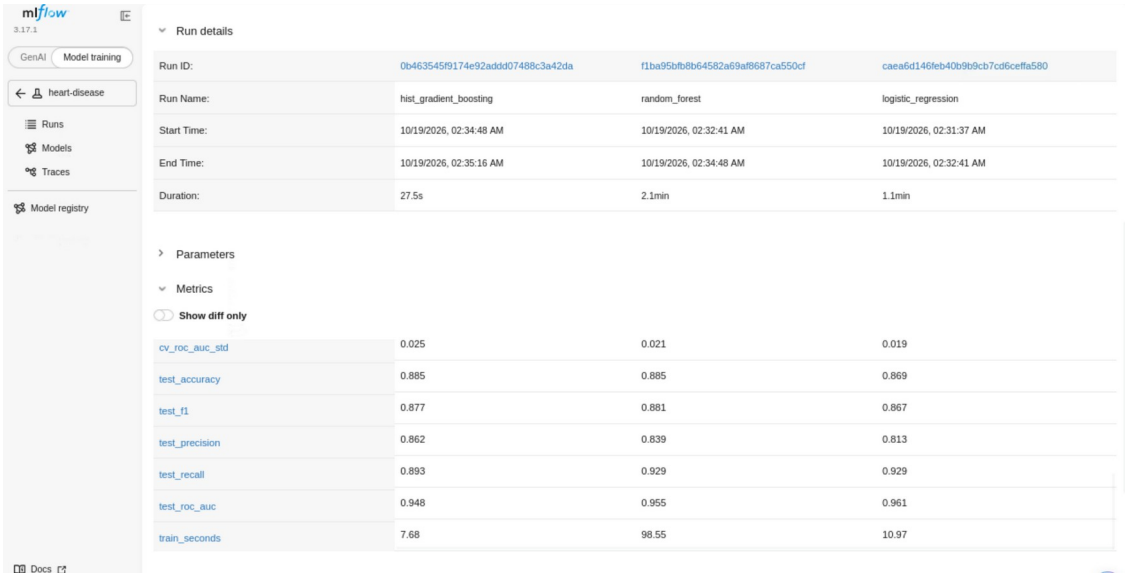


Figure 9. MLflow compare view of the three tuned models

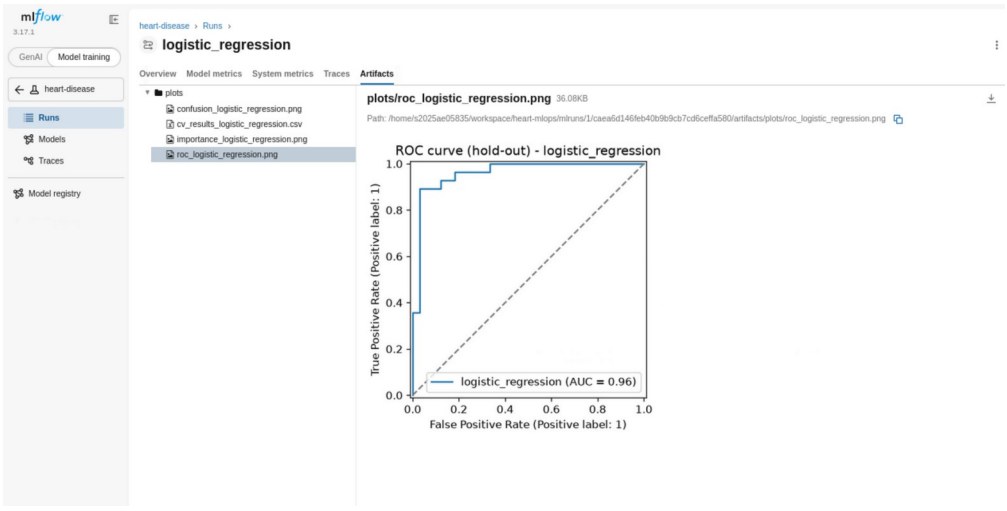


Figure 10. Artifacts logged with the Logistic Regression run

7. Packaging, testing and CI/CD

Packaging and reproducibility

- models/heart_model.joblib holds the full Pipeline (imputers, scaler, encoder, classifier); the API never re-implements preprocessing.
- models/model_metadata.json records model name, MLflow run id, parameters, metrics, feature list, training time and library versions.
- All versions are pinned; fixed random seeds make CV splits and forests deterministic. Re-running the full training on a second machine from the GitHub repository reproduced every CV and hold-out score to three decimals.

Unit tests (18, pytest)

- Data: UCI parsing and '?' handling, mirror-to-UCI mapping, label mapping, impossible values, duplicates, unknown categories.
- Model: no NaNs after preprocessing, scaling, unseen categories, missing optional fields, output contract, better-than-chance AUC, save/load round trip.
- API: /health, /predict response and request-id header, 422 on invalid input, batch endpoint, /metrics exposure. API tests use a small model trained on synthetic data so they do not depend on a prior training run.

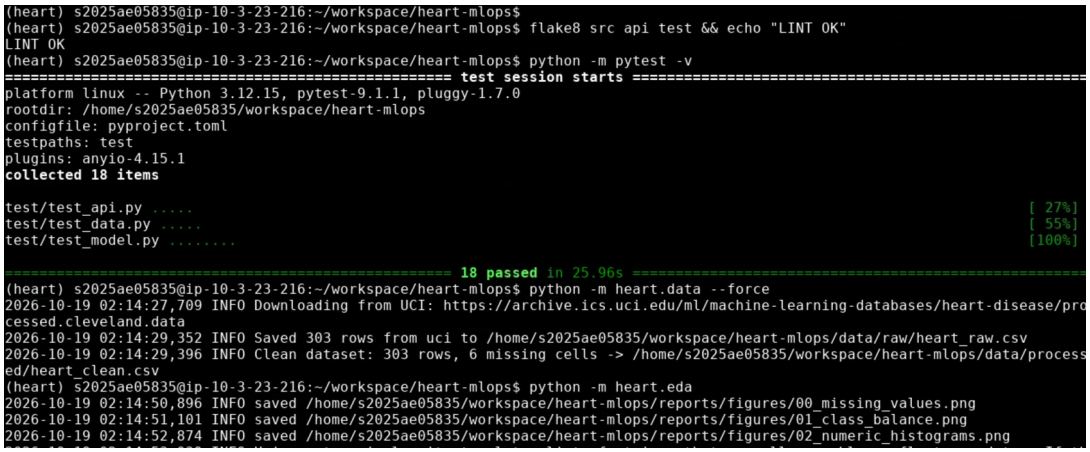


Figure 11. Lint and all 18 unit tests passing locally

GitHub Actions pipeline

Job	What it does	Artifacts
lint	flake8 on src, api, test; errors annotated inline on the PR	—
test	pytest -v with JUnit XML	test-results
train	download → EDA → train; fails if CV ROC-AUC < 0.85	model, reports, training.log, MLflow db
docker	build image, run it, curl /health, /predict, /metrics; push to GHCR on main	prediction.json, container.log

Jobs are chained with needs, so any lint error, failing test or quality-gate miss stops the pipeline with a red run and a clear log.

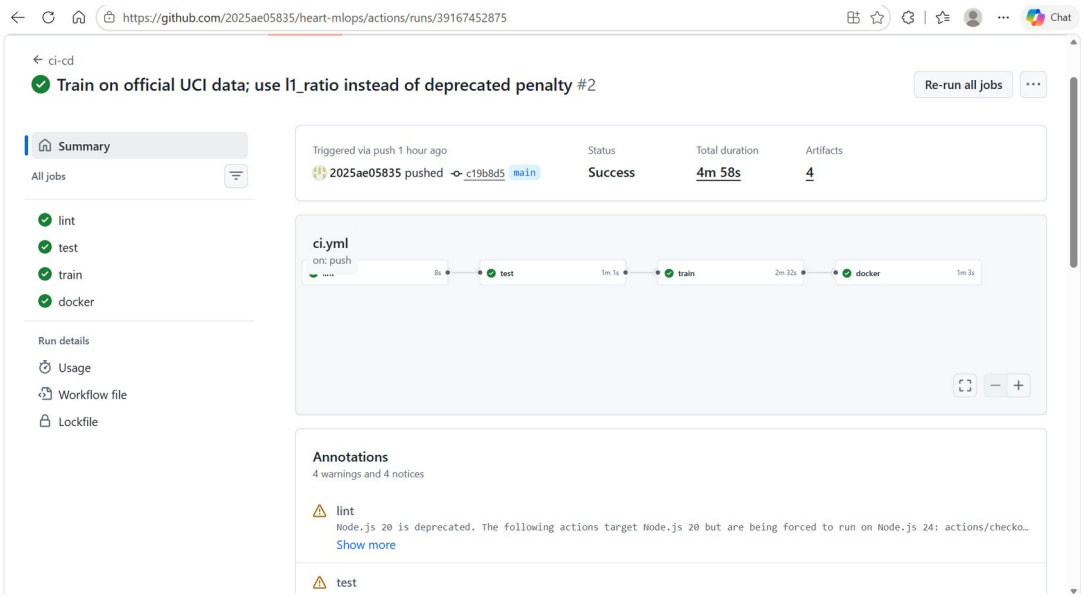


Figure 12. Successful pipeline run on main: lint, test, train, docker (4 min 58 s)

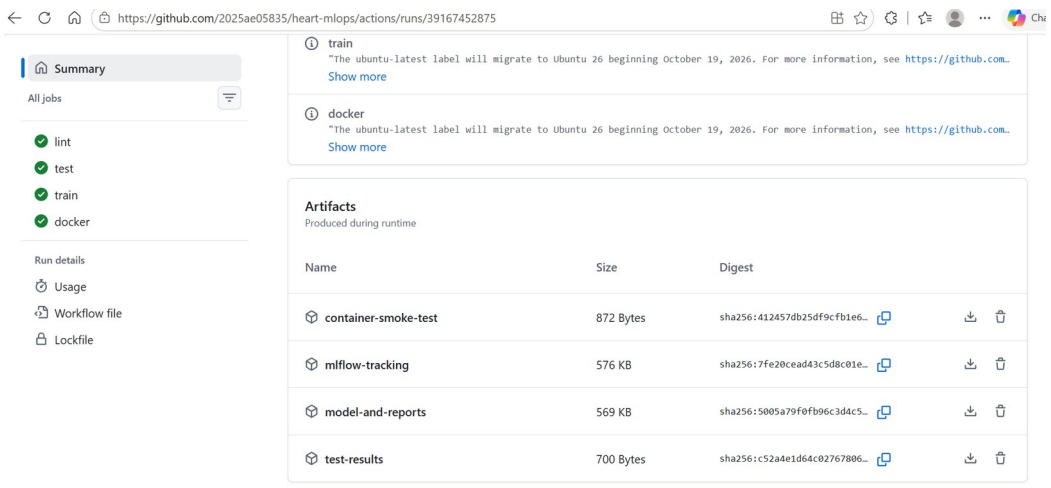


Figure 13. Artifacts stored with the run

To prove that the pipeline blocks bad code, a pull request with a deliberate style error was opened. The lint job failed after 6 seconds, the test, train and docker jobs were skipped, and GitHub marked the pull request as failing; it was closed without merging (Figure 14).

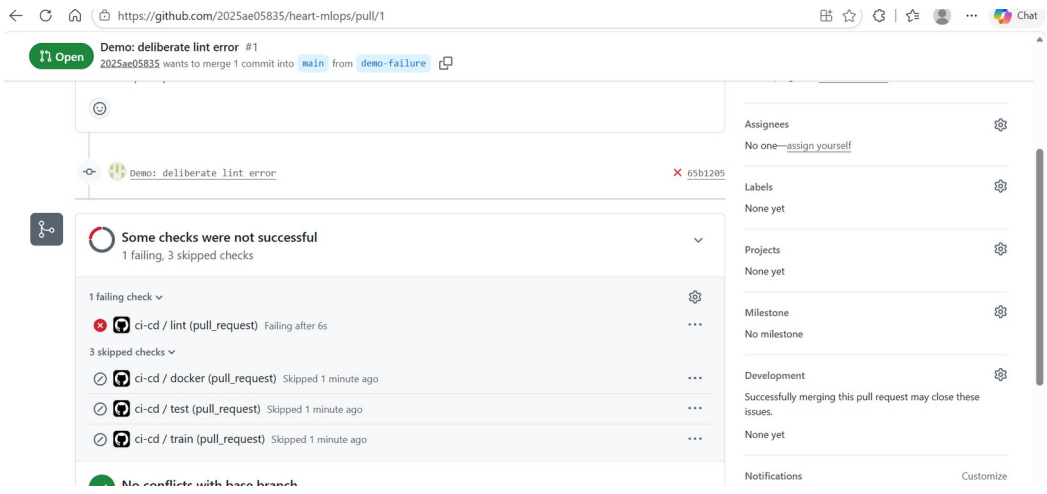


Figure 14. Deliberate lint error in a pull request: pipeline fails, later jobs skipped

8. Containerisation and deployment

Docker image

Multi-stage build on python:3.12-slim: dependencies are installed into a virtualenv in a builder stage and copied into the runtime stage, which runs as a non-root user (uid 10001), exposes port 8000 and has a HEALTHCHECK on /health. One uvicorn worker per container keeps Prometheus counters consistent; scaling is done with replicas.

```
docker build -t heart-api:latest .
docker run --rm -p 8000:8000 heart-api:latest
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d
@sample/patient.json
{"prediction":1,"label":"heart_disease","probability_disease":0.9938,

"confidence":0.9938,"risk_band":"high","model_name":"logistic_regression","re
quest_id":"..."}
```

Input is validated with pydantic: numeric ranges (e.g. age 18–110, cholesterol 80–700) and allowed category values; violations return HTTP 422 with a field-level message. confidence is max(p, 1 − p).

Running the image locally: the container reports healthy, /health returns the loaded model and its metrics, the high-risk and low-risk sample patients get 0.9938 and 0.0298, an invalid request gets HTTP 422, and every request is logged as one JSON line with its latency (Figure 15).

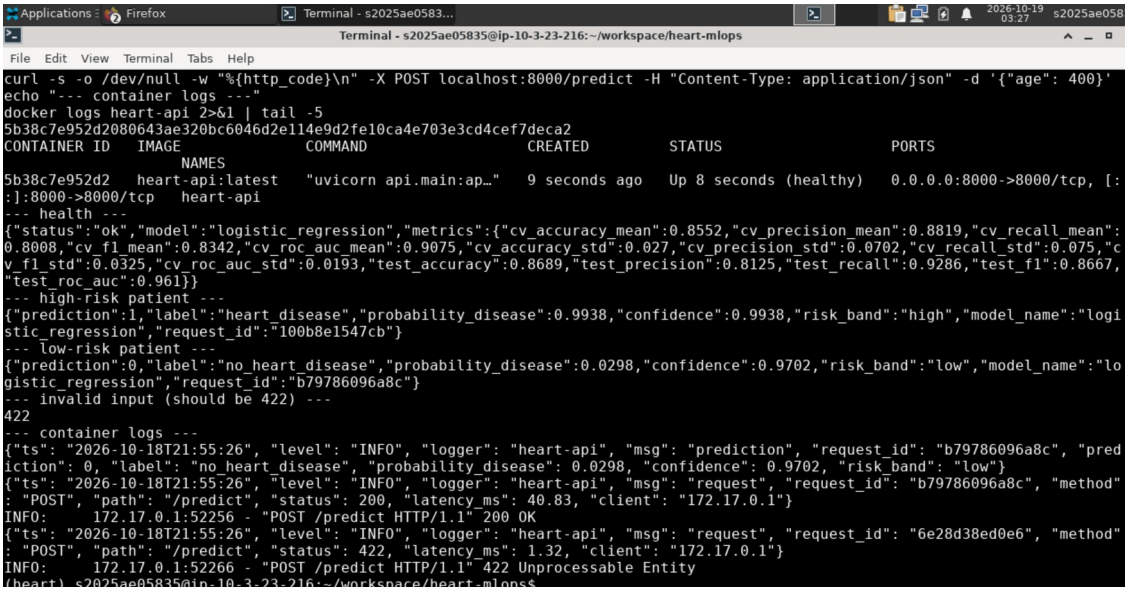


Figure 15. Container test: health, two predictions, input validation and JSON logs

Kubernetes

k8s/deployment.yaml creates namespace heart with a Deployment (2 replicas, rolling update with zero unavailable pods, CPU/memory requests and limits, readiness and liveness probes on /health, non-root security context), a LoadBalancer Service on port 80, and a HorizontalPodAutoscaler (2–5 replicas at 70% CPU). k8s/ingress.yaml optionally exposes the same service through NGINX Ingress at heart.local. scripts/deploy_k8s.sh builds the image inside Minikube and applies everything.

```
minikube start --cpus 4 --memory 6g
bash scripts/deploy_k8s.sh
minikube tunnel                          # separate terminal
kubectl -n heart get pods,svc
curl -X POST http://<EXTERNAL-IP>/predict -H 'Content-Type: application/json'
-d @sample/patient.json
```

On Minikube, minikube tunnel assigns the LoadBalancer the external IP 10.96.111.167. Both API replicas, Prometheus and Grafana run in namespace heart, and /health and /predict answer through the LoadBalancer (Figure 16). The interactive Swagger UI at /docs is shown in Figure 17; note that the x-request-id header matches the request_id in the body, so a response can be traced to its log line.

```
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$ cd ~/workspace/heart-mlops
kubectl -n heart get pods,svc -o wide
EXTERNAL_IP=$(kubectl -n heart get svc heart-api -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "LoadBalancer IP: $EXTERNAL_IP"
curl -s http://$EXTERNAL_IP/health | head -c 120; echo
curl -s -X POST http://$EXTERNAL_IP/predict -H "Content-Type: application/json" -d @sample/patient.json; echo
NAME                                READY   STATUS    RESTARTS   AGE    IP            NODE       NOMINATED NODE   READINESS GATES
pod/grafana-8677db784c-66kvx        1/1     Running   0          3m4s   10.244.0.6    minikube   <none>           <none>
pod/heart-api-79889d8fd8-49zhd      1/1     Running   0          3m5s   10.244.0.4    minikube   <none>           <none>
pod/heart-api-79889d8fd8-k7pt8      1/1     Running   0          3m5s   10.244.0.3    minikube   <none>           <none>
pod/prometheus-785b6b6999-gh9hz     1/1     Running   0          3m5s   10.244.0.5    minikube   <none>           <none>

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP     PORT(S)          AGE    SELECTOR
service/grafana      NodePort       10.106.47.64    <none>          3000:30300/TCP   3m4s   app=grafana
service/heart-api    LoadBalancer   10.96.111.167   10.96.111.167   80:32339/TCP     3m5s   app=heart-api
service/prometheus   ClusterIP      10.106.130.86   <none>          9090/TCP         3m5s   app=prometheus
LoadBalancer IP: 10.96.111.167
{"status":"ok","model":"logistic_regression","metrics":{"cv_accuracy_mean":0.8552,"cv_precision_mean":0.8819,"cv_recall_
{"prediction":1,"label":"heart_disease","probability_disease":0.9938,"confidence":0.9938,"risk_band":"high","model_name":"logi
stic_regression","request_id":"634703907247"}
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$ curl -s -X POST http://$EXTERNAL_IP/predict -H "Content-Type: app
lication/json" -d @sample/patient.json; echo
{"prediction":1,"label":"heart_disease","probability_disease":0.9938,"confidence":0.9938,"risk_band":"high","model_name":"logi
stic_regression","request_id":"f33f41c90c07"}
(heart) s2025ae05835@ip-10-3-23-216:~/workspace/heart-mlops$
```

Figure 16. Pods, services (LoadBalancer with external IP) and predictions through the LoadBalancer

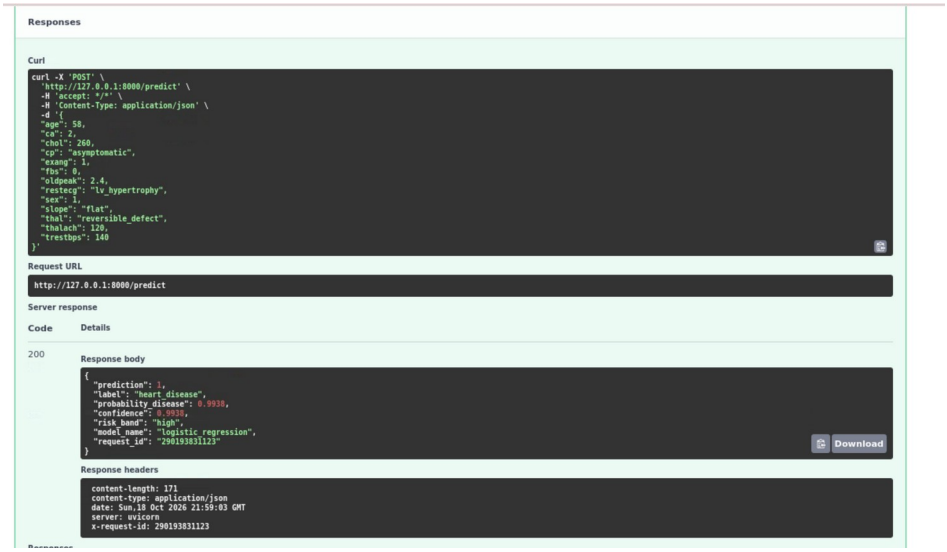


Figure 17. Swagger UI: POST /predict with HTTP 200 response

9. Monitoring and logging

Every request is logged to stdout as one JSON line (timestamp, request id, method, path, status, latency, client), and each prediction is logged with its probability and risk band. JSON logs can be read with kubectl logs and are ready for a log collector such as Loki or ELK. The request id is returned in the x-request-id header so a client error can be traced to a log line.

/metrics exposes Prometheus metrics: request counts by endpoint and status, a latency histogram, prediction counts by label and risk band, a histogram of predicted probabilities and a model-info gauge. In Kubernetes, Prometheus discovers the API pods through their prometheus.io annotations; Grafana is provisioned automatically with the datasource and a dashboard showing request rate, error rate, p50/p95/p99 latency and the distribution of predictions. A shift in the risk-band mix is a simple, label-free signal of data drift.

```
python scripts/load_test.py --url http://<API_URL> --n 300   # 5%
deliberately invalid
kubectl -n heart port-forward svc/grafana 3000:3000
```

Prometheus found both API pods automatically and scrapes each one separately (2/2 up, Figure 18), so counters are never mixed when the LoadBalancer spreads traffic. A load test of 1,500 requests, 5% of them deliberately invalid, produced the dashboard in Figure 19: 1,425 predictions, an error rate of 4.6% (the invalid requests, all rejected with HTTP 422) and a p95 latency of about 38 ms. The probability-distribution panel shows Prometheus histogram buckets, which are cumulative (each value counts predictions at or below that probability).

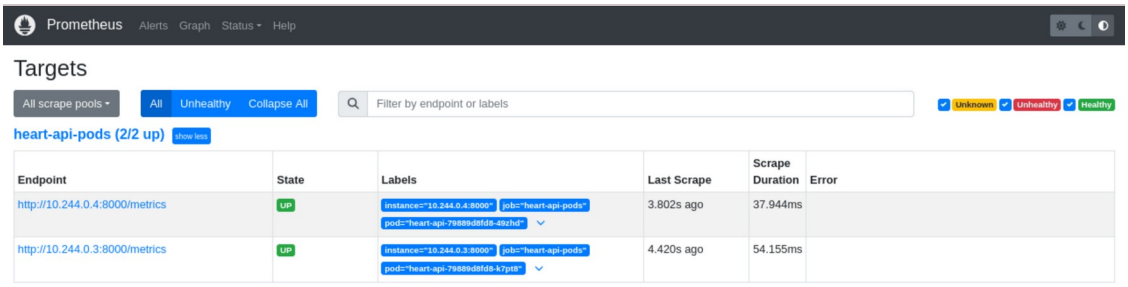


Figure 18. Prometheus targets: both heart-api pods discovered and UP

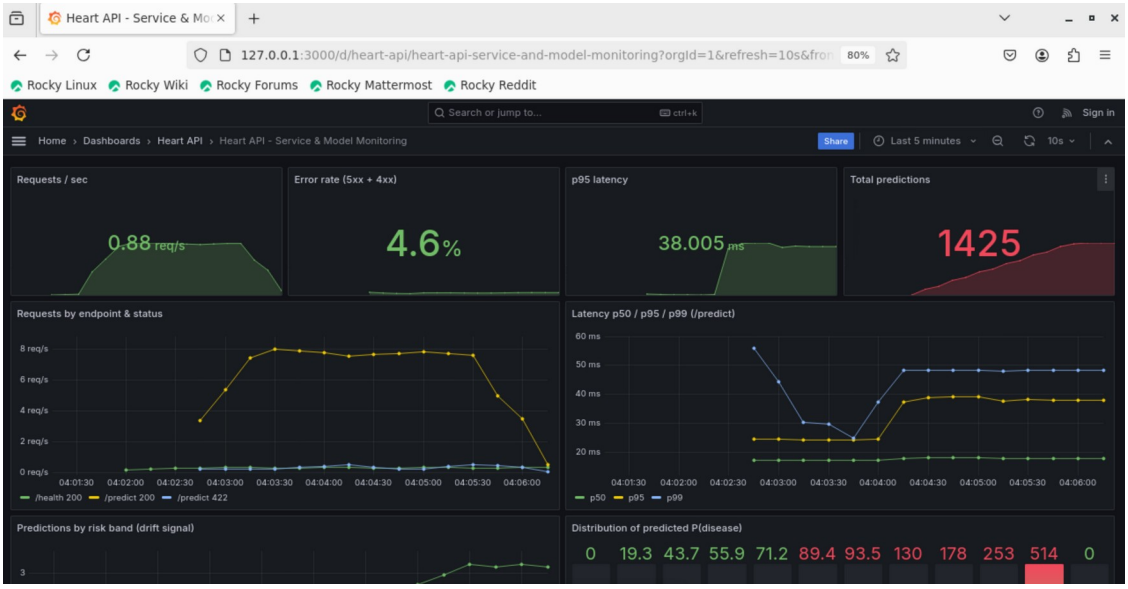


Figure 19. Grafana dashboard during the load test

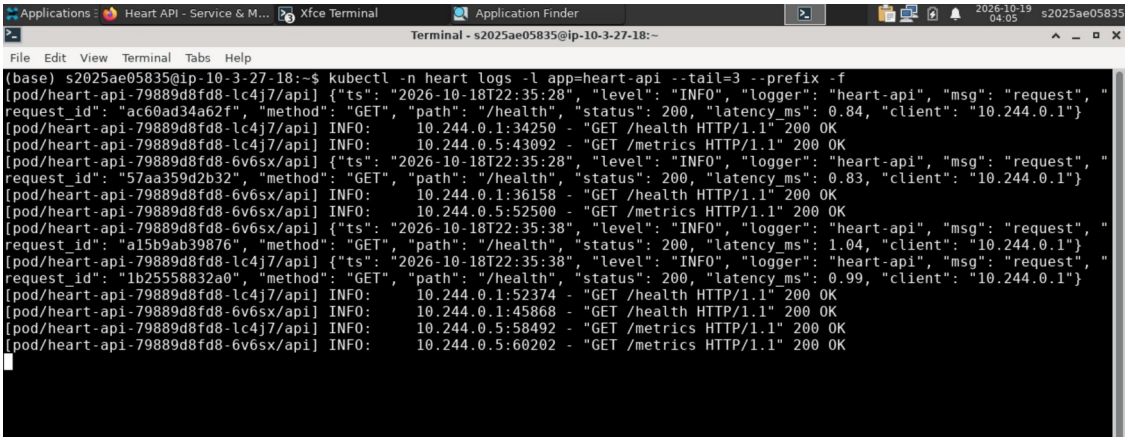


Figure 20. Structured JSON request logs from both pods (kubectl logs)

10. Limitations and next steps

- 303 patients from a single hospital: performance estimates have wide uncertainty and may not transfer to other populations. The other UCI sites (Hungary, Switzerland, VA) could be used for external validation.
- Decision threshold is fixed at 0.5; a recall-oriented threshold should be chosen with clinicians.
- MLflow runs locally; a shared tracking server with object storage would support team use.
- Monitoring covers service health and prediction distribution; ground-truth feedback would allow tracking real accuracy over time.

11. Links

- Code repository: https://github.com/2025ae05835/heart-mlops
- Container image: ghcr.io/2025ae05835/heart-api:latest
- API access: local Minikube; run scripts/deploy_k8s.sh and minikube tunnel, then call http://<EXTERNAL-IP>/predict (Section 8)
- Video walkthrough: https://github.com/2025ae05835/heart-mlops/blob/main/video/2025ae05835.mp4